

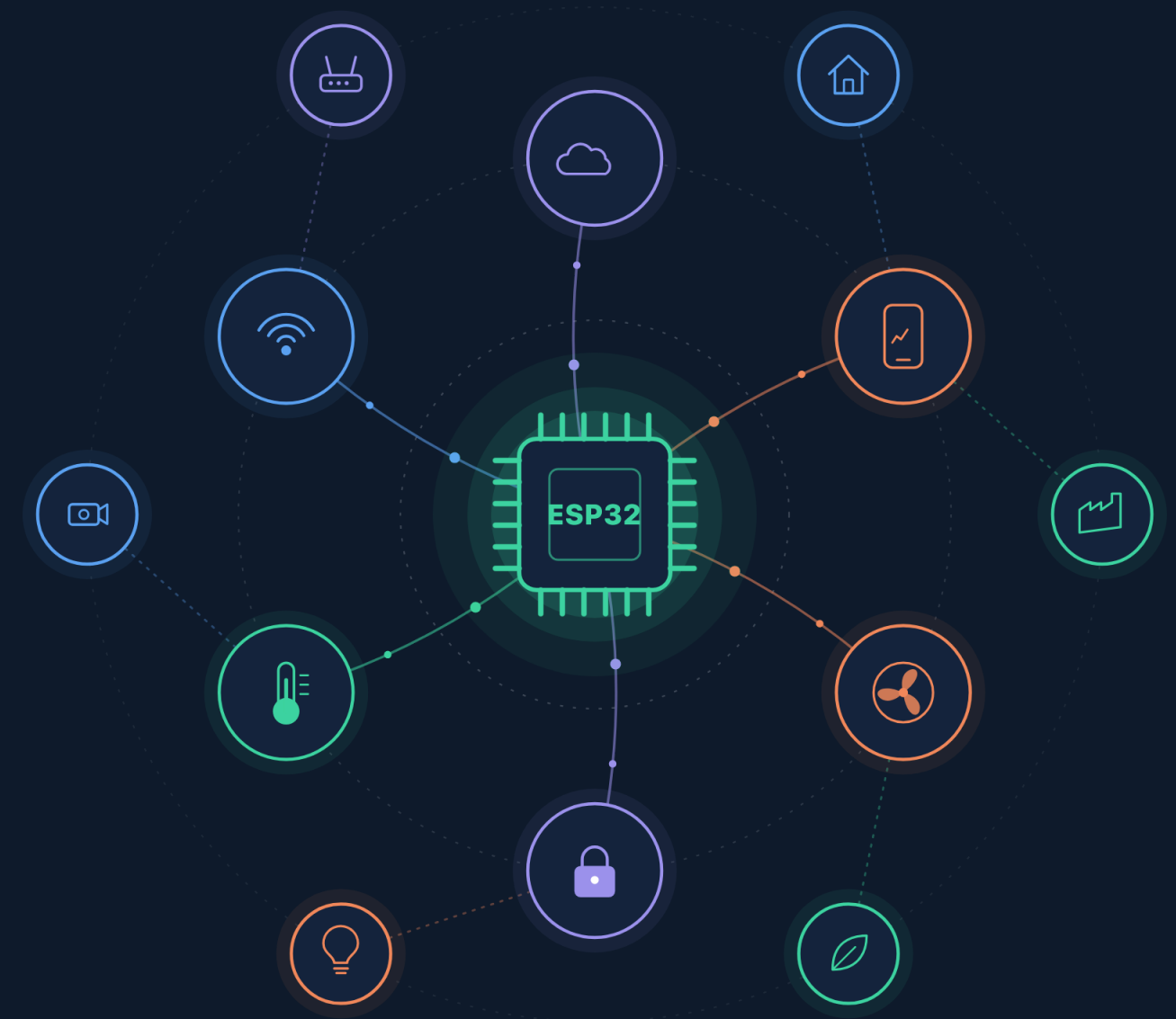
MODULE 3 • 45 MINUTES

Sensors, Actuators, and Reading Data

How a device feels the room, switches on a fan, and turns signals into numbers.

Mehdi Lotfinejad

Introduction to the Internet of Things



THE ROAD AHEAD

Eight modules, one smart device

PART 1 — DEVICES AND DATA

01

What is IoT?

Lecture

02

How it's built

Lecture

03

Sensors and actuators

You are here

04

Getting online

Lab 3

PART 2 — CLOUD, AUTOMATION AND SECURITY

05

Cloud and dashboards

Lab 4

06

Automation

Lab 5

07

Security

Group activity

08

Real world + capstone

Lab 6

BY THE END OF THIS MODULE

You will be able to...

01 Describe five common sensors and four common actuators

02 Explain the difference between analog and digital signals

03 Explain how a microcontroller turns a signal into a number

04 Read a short Arduino program and say what each part does

05 Connect and read a real sensor in the Wokwi simulator

THE BIG PICTURE

The journey of one reading



Real world

The room is 26 °C.



Sensor

Turns heat into an electrical signal.



Pin

The microcontroller reads the signal.



Your code

Converts the signal into a number.



26.0 °C

Ready to use or to send.

Actions go the other way: code decides → a pin sends a signal → an actuator acts.

Part A

Sensors: how devices feel the world

Five common types, and what each one measures.

FIVE COMMON SENSORS

What each sensor measures



Temperature

How hot or cold. A thermistor, for example.



Humidity

Water vapour in the air, as a percentage.



Light

How bright. A photoresistor, or LDR.



Motion (PIR)

A warm body moving past.



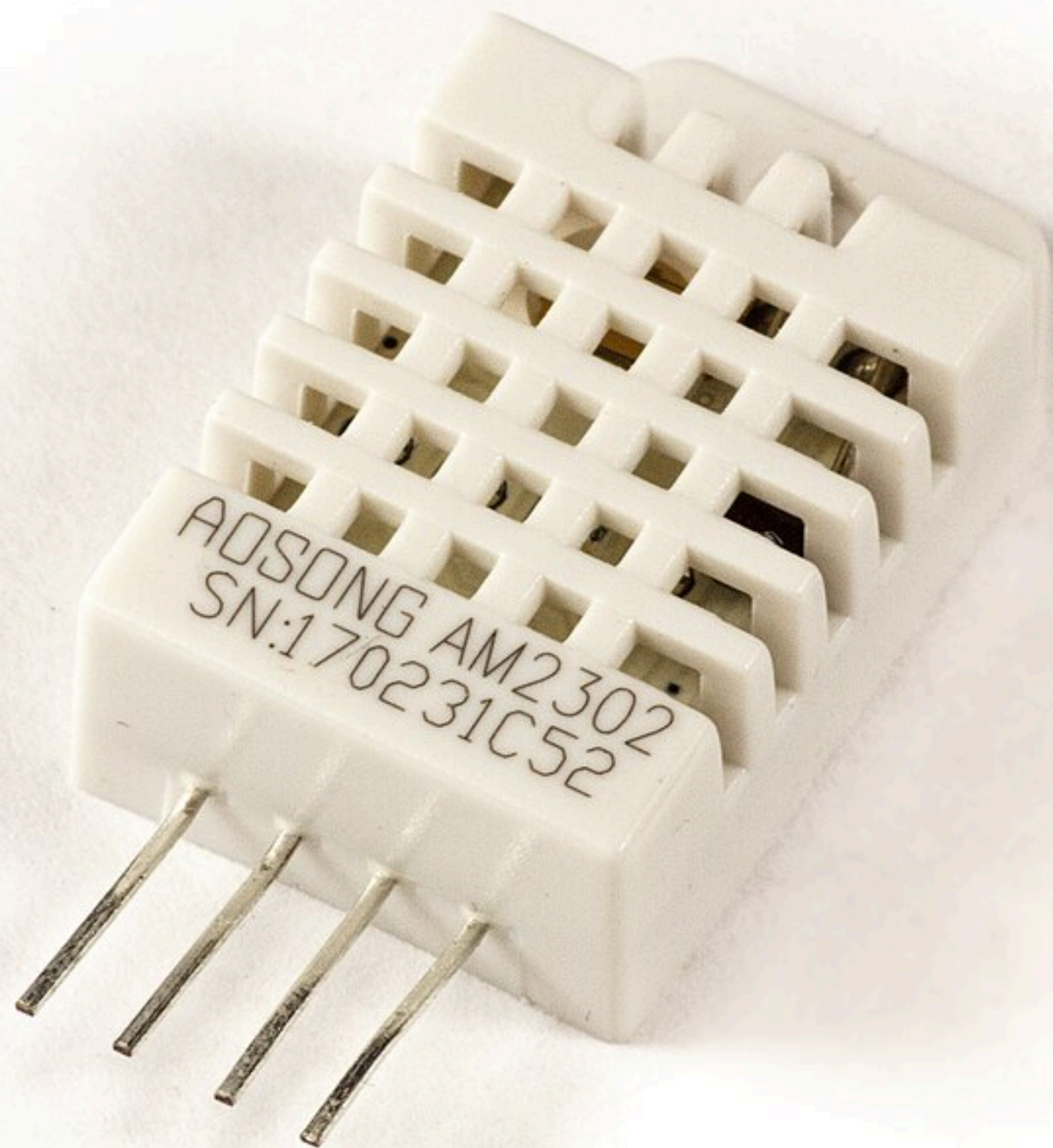
Gas

Gases like carbon monoxide or LPG.

TWO SENSORS IN ONE

The DHT22

- Temperature **and** humidity in one small part
- Also sold as the AM2302
- The sensor you will use in **Lab 2**



Part B

Actuators: how devices act

Light, sound, switching and movement.

FOUR COMMON ACTUATORS

From a signal to an action



LED

A small light. Needs a resistor. Great for status lights.



Buzzer

A beep or a tone, to get attention.



Relay

A switch for a bigger circuit: a fan, a pump.



Motor

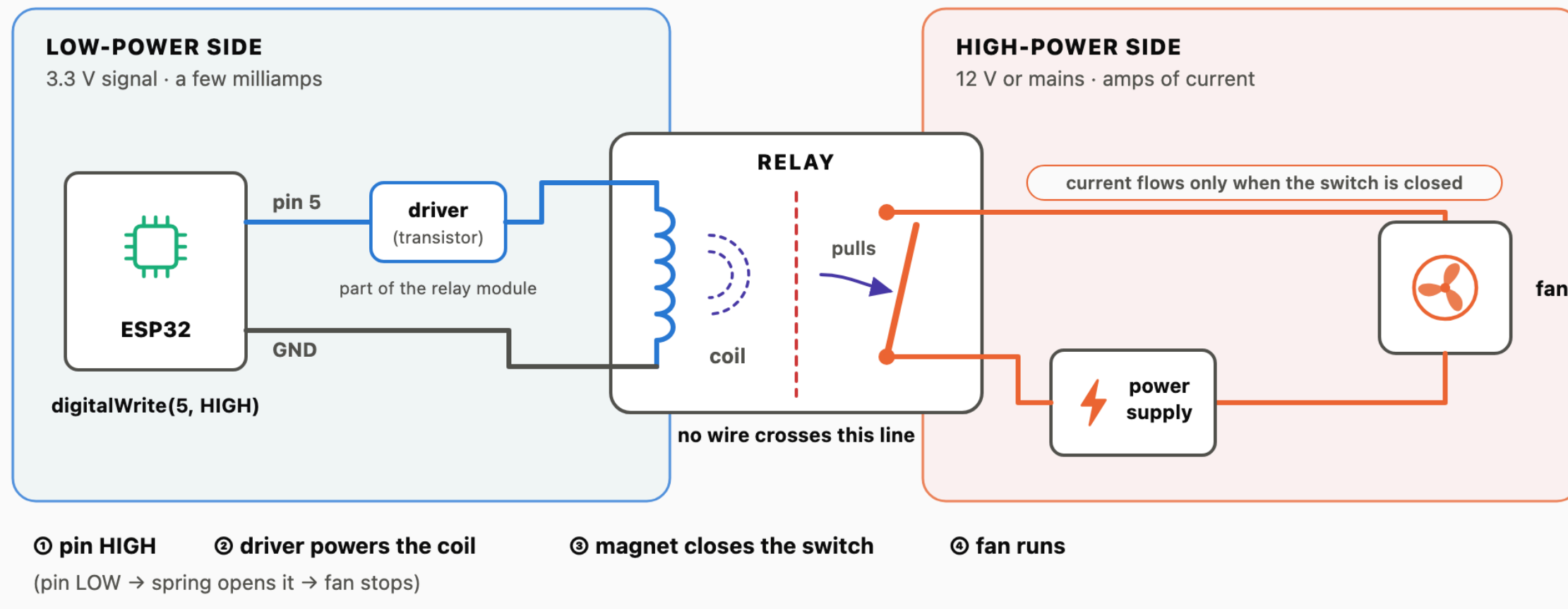
Movement. DC, servo or stepper, via a driver.

HOW A RELAY WORKS

A small signal switches a big circuit

How a relay lets a tiny chip switch a big load

Two separate circuits. The small one energises a coil; its magnetism closes the big one's switch.



A pin gives only a **small** amount of power

The coil's **magnetism** closes the switch

The two circuits stay **separate**

A REAL PART

A relay module

- A typical **5 V** relay module
- Screw terminals connect the high-power circuit
- Fans, pumps, heaters, lights
- **Mains electricity can kill.** The labs are simulated.



PAUSE AND CHECK

You want your room monitor to open a window vent to exactly half-way. Which actuator would you choose?

LED, buzzer, relay, or motor — and which kind of motor?

Part C

Analog vs digital signals

The most important idea in this module — and simpler than it sounds.

TWO KINDS OF SIGNAL

Digital and analog, side by side

	Digital	Analog
Values	Two: HIGH/LOW, 1/0, on/off	Any value in a range
Like a...	Light switch	Dimmer switch, volume knob
Sensors	Push button, PIR sensor	Potentiometer, light sensor, thermistor
Read with	digitalRead()	analogRead()
It answers	“Is it happening?”	“How much?”

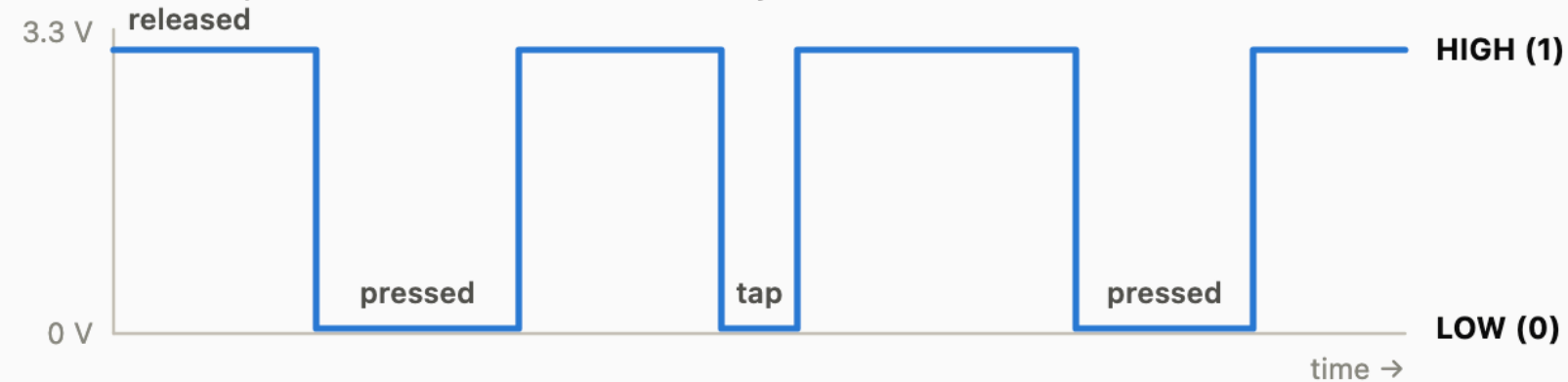
SEEING THE DIFFERENCE

From a voltage to a number

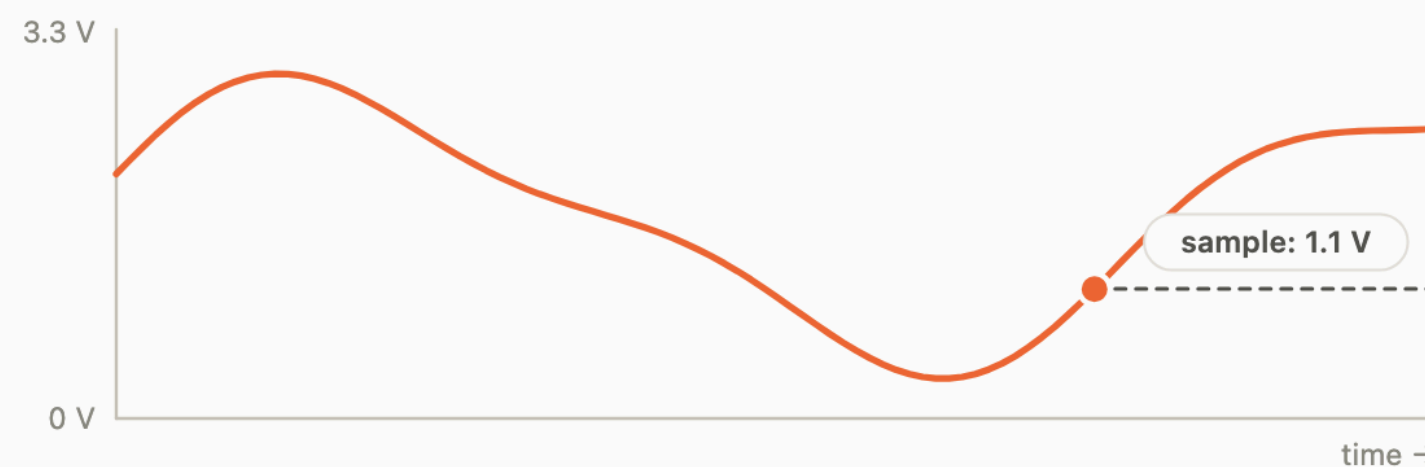
Digital vs analog — and how the ESP32 reads analog

Digital has two values. Analog can be anything in a range; the ADC measures it like a ruler.

DIGITAL a push button wired as in Lab 1: only HIGH or LOW

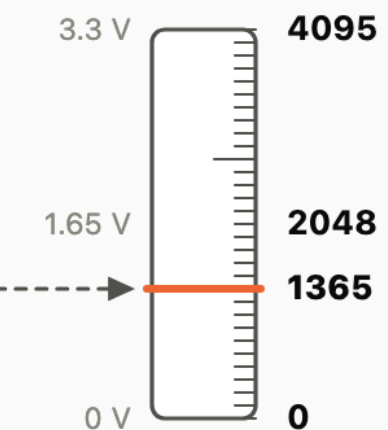


ANALOG a knob / light sensor: any value in between



THE ADC "RULER"

12-bit → 4,096 marks



then your code does:

`analogRead() → 1365`

$1365 \div 4095 \times 100$

= 33.3 %

THE ESP32'S 12-BIT ADC

`analogRead()` gives 0 to 4095

0

at 0 V (nothing)

≈ 2048

at about 1.65 V (half-way)

4095

at about 3.3 V (full)

Read a raw number, then **convert** it: $(\text{raw} \div 4095) \times 100 = \text{percent}$. Use **ADC1 pins (GPIO 32–39)** with Wi-Fi.

Six rules for connecting parts



Three wires

Power, ground and signal.



Match voltage

The ESP32 works at 3.3 V.



No floating

Use a pull-up resistor on inputs.



Use a library

Ready-made code for smart sensors.



Respect limits

DHT22: one reading every 2 seconds.



Smooth noise

Average several readings.

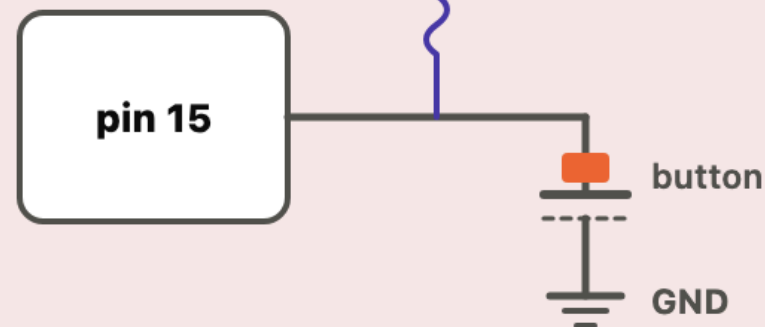
RULE THREE

Don't leave inputs floating

Why buttons need a pull-up resistor

An unconnected input "floats" and reads random noise. A pull-up holds it steady until pressed.

❌ **Floating input — no pull-up**
picks up stray noise
like an antenna

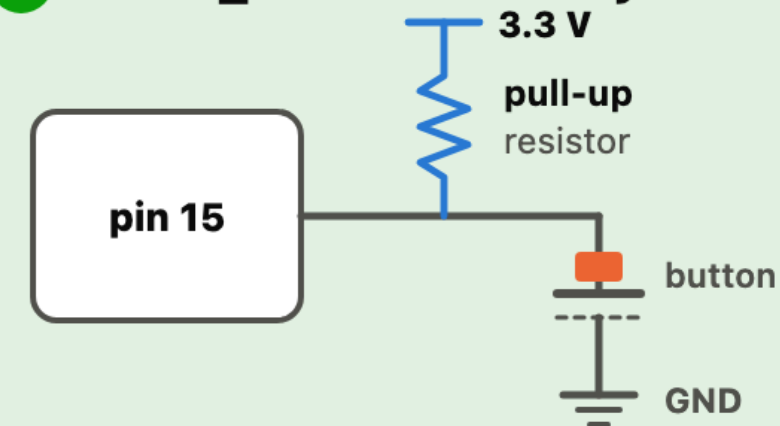


What digitalRead() returns, reading after reading:

H L H H L H L L H L H H L H

Random — the code sees presses that never happened.

✅ **INPUT_PULLUP — steady and reliable**



What digitalRead() returns, reading after reading:

H H H H H L L L L H H H H H

button held down

HIGH at rest, LOW only while pressed.

Code: `pinMode(15, INPUT_PULLUP);`

PAUSE AND CHECK

Your button randomly shows “pressed” even when nobody is touching it. What’s the most likely cause, and the fix?

Think about the picture we just saw.

setup() once, loop() forever

Command	What it means
<code>pinMode(2, OUTPUT);</code>	“I’m going to control pin 2.”
<code>digitalWrite(2, HIGH);</code>	“Switch the LED on.”
<code>digitalRead(15)</code>	“Is the button pressed?”
<code>analogRead(34)</code>	“How far is the knob turned?” (0–4095)
<code>delay(2000);</code>	“Wait two seconds.”
<code>Serial.println("Hi");</code>	“Show ‘Hi’ in the Serial Monitor.”

Anything after `//` is a **comment**: a note for humans. The computer ignores it.

COMMON MISCONCEPTIONS

Four ideas that trip people up

✗ Myth *Digital is always more accurate than analog.*

✓ Fact They are types of signal, not levels of quality.

✗ Myth *A pin can power anything directly.*

✓ Fact Fans, motors and pumps need a relay or a driver.

✗ Myth *If the code shows a number, it must be right.*

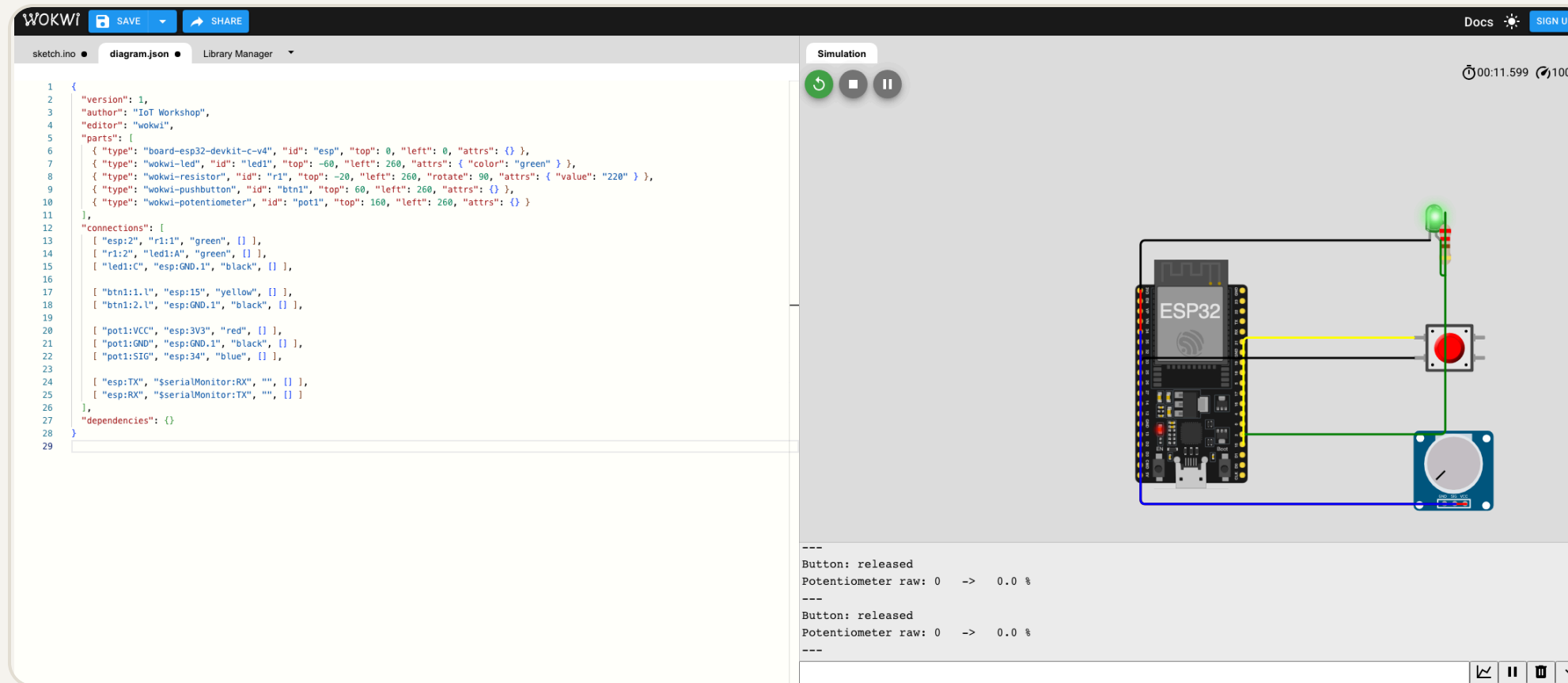
✓ Fact Always ask: is this number sensible?

✗ Myth *A pressed button always reads HIGH.*

✓ Fact With INPUT_PULLUP, a pressed button reads LOW.

LAB 1 · ABOUT 25 MINUTES

Digital vs analog in Wokwi



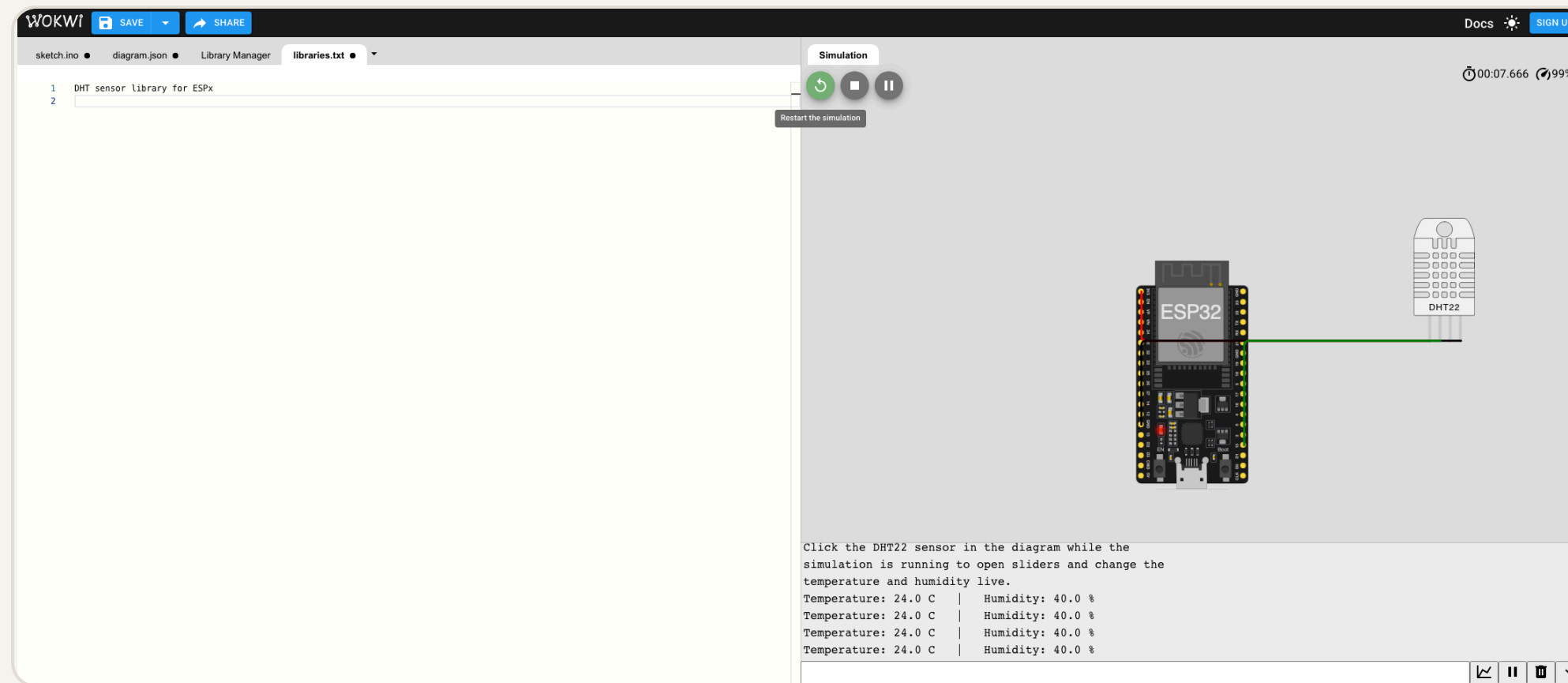
- 1 Open a new ESP32 project in Wokwi
- 2 Paste sketch.ino, then the whole diagram.json
- 3 Press **Play** and watch the LED blink
- 4 Hold the button: the monitor says PRESSED
- 5 Drag the dial: 0 to 4095, shown as a percent



Scan for the step-by-step guide
github.com/datatweets/intro-to-iot-workshop

LAB 2 • ABOUT 25 MINUTES

Reading a DHT22 sensor



- 1 Open a new ESP32 project, or reuse Lab 1
- 2 Paste sketch.ino and diagram.json
- 3 Add **libraries.txt** with the DHT library name
- 4 Press **Play**; the first build takes longer
- 5 Click the DHT22 and drag its sliders



Scan for the step-by-step guide
github.com/datatweets/intro-to-iot-workshop

KEY TAKEAWAYS

What to remember

- ✓ **Sensors** turn the physical world into electrical signals
- ✓ **Actuators** act; relays and motors need helper circuits
- ✓ **Digital** = two states. **Analog** = any value in a range
- ✓ The **ADC** gives 0–4095; your code **converts** it to real units
- ✓ **setup()** runs once; **loop()** runs forever

Quick quiz

Q1 Name one sensor with a digital signal and one with an analog signal.

Q2 On an ESP32, `analogRead()` returns 1024. About what percent is that?

Q3 Why can't you connect a fan straight to an ESP32 pin?

Q4 Your code reads a DHT22 ten times a second. Why are the readings strange?

UP NEXT • MODULE 4

Getting Online

Connectivity and communication protocols: Wi-Fi, Bluetooth, LoRaWAN and MQTT — and your DHT22 readings sent across the internet in Lab 3.



Mehdi Lotfinejad

Book, labs and slides: github.com/datatweets/intro-to-iot-workshop